

VAJRA prototype build brief

Smart India Hackathon 2026, problem statement PS26051. Specification and agent instructions for building the working prototype. Prepared 3 October 2026.

How to use this document. Give an AI coding agent this PDF together with the master prompt in section 11. The prompt is also supplied as a separate text file (VAJRA_master_prompt.txt) so it can be pasted without broken lines. The agent reads this brief first, then builds in the order set out in section 9. Sections 1 to 7 say what to build. Section 8 says how it must look and read, and why. Sections 9 and 10 say how to work and when to stop.

Section	Contents
1	What the problem statement asks, and the gap to close
2	The solution and its scope, with architecture
3	Physics model specification
4	Data: weather and materials
5	Outputs, metrics and the optimiser
6	Verification and validation tests
7	ANSYS bridge
8	AI slop: research findings, rules, design specification, automated scan
9	Repository layout, data contract, milestones with gates
10	Iteration protocol and stop rules
11	Master prompt
12	Sources

1. What the problem statement asks

PS26051 asks for a software model that predicts how warm the inside of a shelter will be, given the weather outside and the shelter's design, so that a passive shelter (one that needs no heater) can be designed for a particular region. Ladakh is the worked example. It has a lot of sun (1900 to 2100 kWh/m² a year, about 7.9 hours of sunshine a day, more than 300 clear days a year), so shelters warm up in the daytime. After sunset the inside drops close to the outside temperature, because heat escapes through the walls, roof, floor and openings. People then burn fuel to stay warm. The model must also work for other climates by changing the inputs.

The table splits the statement into separate requirements and says what the prototype does about each one and how we will know it works.

Requirement in the problem statement	What the prototype does	Check
Thermal analysis of the shelter: size, shape, orientation	Rectangular plan (length, width, wall height), flat, mono-pitch or gable roof, orientation angle. Reports surface area to volume ratio.	T7
Materials, thermal mass storage, composite multi-material	Walls, roof and floor are stacks of layers, each with a material and thickness. Library covers masonry, insulation, timber, water and a phase change material.	T1, T5, T6
Effect of openings	Windows per facade with glazing type and an optional insulated night cover. Ventilation rate with a day and night schedule.	T7
User friendly, works on user defined values (real time data, material properties)	Weather by latitude and longitude from NASA POWER, an uploaded CSV, or the bundled Leh file. Custom materials can be added in the interface.	T9
Compare materials under the same ambient condition and predict the most efficient combination	Compare view runs several designs on one weather file. The optimiser ranks designs by heating energy needed, with overheating as a second measure.	T8

Requirement in the problem statement	What the prototype does	Check
Predict shelter inside temperature from user inputs	Hourly indoor air temperature for the chosen period.	T2, T3, T10
Predict thermal energy generated from solar radiation	Solar energy on each facade and the roof (kWh/m ²), transmitted through glazing (kWh), absorbed by opaque surfaces (kWh), per day and in total.	T4
Heat flow details against the ambient to shelter temperature difference over a defined period	Heat flow time series for walls, roof, floor, glazing and ventilation, shown with the temperature difference, plus totals in kWh.	T2
Minimise energy use for thermal comfort and fossil fuel in general	Every design is also run with an ideal heater that holds the setpoint. The heater energy in kWh is the headline energy figure, labelled as a model estimate.	T2, T8

The gap to close

The problem statement says the general model should be developed in ANSYS. Our core is a Python solver, and the deck says ANSYS is used to check the finalists. A judge can fairly ask whether ANSYS is really used. Section 7 makes it real: the prototype exports boundary conditions and an APDL input for ANSYS, and the team runs at least one case in ANSYS and compares it with the Python result. An agent cannot run ANSYS for you, so that step belongs to the team, and the comparison should be shown honestly whatever the difference turns out to be.

2. The solution

A local web app on top of a Python package. The user picks a place (or uploads weather), describes a shelter, and sees hourly indoor temperature, solar gain and heat flow. A compare view runs many designs on the same weather and ranks them. The chosen design can be exported as a one page design sheet and as ANSYS input. Everything runs on one laptop with no internet needed once the weather file is downloaded.

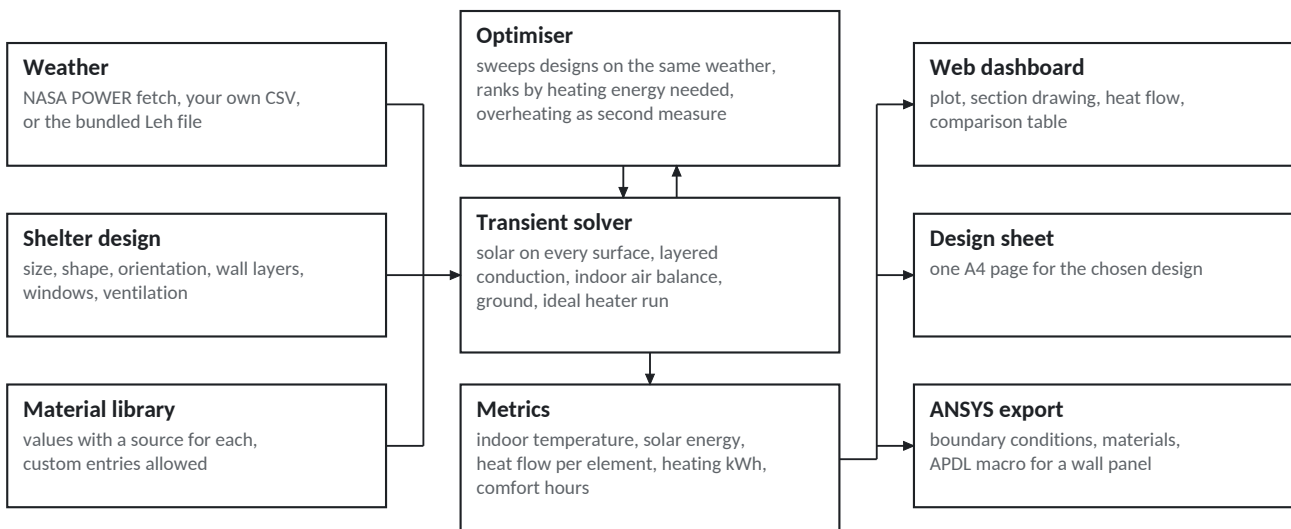


Figure 1. Data flow. Plain boxes and lines on purpose: the diagram should look like something drawn for the report, not a product graphic.

In scope

- Transient heat balance solver: layered conduction, indoor air, solar, sky radiation, ventilation, ground, ideal heater run.
- Weather loader for NASA POWER and user CSV files.
- Material library with a source for every value.
- Parameter sweep and ranking with a named baseline design.
- Web interface, one page design sheet, ANSYS export.
- A validation script ready for sensor data from a test box.

Out of scope for the prototype

- Machine learning. The model is physics based, and adding ML for show would weaken the answer to the judges' obvious question.
- CFD or airflow patterns inside the room. Ventilation is a user rate. ANSYS Fluent stays future work.
- Moisture, condensation, snow load, structural design.
- Any cost, payback, accuracy or fuel saving figure.
- User accounts, databases, cloud hosting.

3. Physics model specification

The solver is a transient thermal network. It cannot be a lookup table or a steady state calculation, because the whole question is what happens after sunset to the heat stored during the day. Values in this section are defaults; each one is an input the user can change, and each default that is an assumption (not a published value) must be marked as an assumption in the interface help text.

3.1 Geometry and surfaces

Surfaces are four walls, the roof (one or two planes) and the floor. Each has an area, tilt, azimuth and construction. Window areas are subtracted from the wall they sit in. The main facade is the one with the glazing and its azimuth is the orientation input (180° is south, measured clockwise from north). The site is in the northern hemisphere by default; the code must not assume it, so that the Southern Hemisphere works from the latitude sign.

3.2 Conduction through layers

Each opaque surface is a one dimensional stack of layers:

$$\rho c \partial T / \partial t = \partial / \partial x (\lambda \partial T / \partial x)$$

Discretise with finite volumes: at least two nodes per layer and node spacing no larger than 25 mm. Integrate in time with backward Euler (implicit), which is stable at any step. Default step 10 minutes, with the hourly weather interpolated linearly between values. For a phase change material use an apparent heat capacity, $c_{\text{eff}}(T) = c + L / \Delta T_m$ inside the melting range ΔT_m , and iterate until the temperature change between iterations is below 0.01 K. Compare this against an enthalpy method as one of the logged experiments in section 10.

3.3 Outside surface

Heat exchange at the outer face has three parts: convection to the air, absorbed solar radiation, and longwave exchange with the sky and ground.

$$q = h_c (T_{\text{air}} - T_s) + \alpha I_t + \epsilon \sigma (F_{\text{sky}} T_{\text{sky}}^4 + F_{\text{gnd}} T_{\text{gnd}}^4 - T_s^4)$$

Convective coefficient $h_c = 5.7 + 3.8 v$ with v in m/s. Scale the 10 m wind from the weather file down to the surface with a power law (exponent 0.14 unless the user sets a terrain). Sky temperature comes from the downwelling longwave in the weather file if it is there, and otherwise from $T_{\text{sky}} = 0.0552 T_{\text{air}}^{1.5}$ (kelvin). The sky view factor of a surface is $F_{\text{sky}} = (1 + \cos \beta) / 2$ with β the tilt, and the ground sees the rest. Linearise the radiation term with $h_r = 4 \epsilon \sigma T_m^3$. This night time radiative loss matters a great deal on a clear high altitude night, so it must not be dropped.

3.4 Inside surface

Use the combined inside surface resistances of ISO 6946: 0.13 m²K/W for horizontal heat flow, 0.10 for upward and 0.17 for downward. They are also what the steady state check T1 uses. Radiation between interior surfaces is folded into these combined values for the prototype.

3.5 Solar

Use pvlib for the solar position, computed at the middle of each weather interval, and for transposing the global, direct and diffuse irradiance onto every surface (the Hay-Davies model is fine). Ground reflectance defaults to 0.2 and is user adjustable, because snow raises it a lot and Ladakh winters have snow. Glazing transmits $\text{SHGC} \times A \times I_t \times \text{IAM}(\theta)$, with the incidence angle modifier $\text{IAM} = 1 - b_0 (1/\cos \theta - 1)$ and $b_0 = 0.1$. Of the transmitted solar energy, 70% is absorbed by the floor and interior mass surfaces and 30% is spread over the other interior surfaces by area. That split is an assumption: expose it as an input and flag it.

3.6 Indoor air

$$C_{air} \frac{dT_{air}}{dt} = \sum h_i A_k (T_{s,k} - T_{air}) + (\rho c_p V N / 3600) (T_{out} - T_{air}) + Q_{int} + Q_{heat}$$

N is air changes per hour, the sum of infiltration and any scheduled ventilation. Air density comes from the surface pressure in the weather file, $p = P / (287.05 T)$. At Leh's altitude the pressure is roughly a third lower than at sea level, so the density is too, and hard coding 1.2 kg/m^3 would overstate ventilation loss by about a half. Provide an optional capacity multiplier on the air node for furnishings, default 1.

3.7 Ground, heater run, start-up

The floor couples to a one dimensional soil column (default 2 m deep, $\lambda = 1.5 \text{ W/mK}$, $\rho = 1800 \text{ kg/m}^3$, $c = 1000 \text{ J/kgK}$, all marked as assumptions) with the deep boundary held at the annual mean air temperature of the weather file. Run every design twice. The free-floating run has no heater. The heater run adds exactly the power needed to hold the indoor air at the setpoint, and its total energy is the headline energy metric. Do not model a real heater, fuel or efficiency. Repeat the first three days of the weather as warm-up and discard them from the results.

4. Data specification

4.1 Weather

Quantity	NASA POWER parameter	Used for
Air temperature at 2 m	T2M	outside air temperature
Relative humidity at 2 m	RH2M	shown only, no moisture model
Wind speed at 10 m	WS10M	convection, scaled to the surface
Surface pressure	PS	air density
Global horizontal irradiance	ALLSKY_SFC_SW_DWN	solar
Diffuse horizontal irradiance	ALLSKY_SFC_SW_DIFF	solar split
Direct normal irradiance	ALLSKY_SFC_SW_DNI	solar split
Downwelling longwave	ALLSKY_SFC_LW_DWN	night sky exchange

Confirm every parameter name and unit in the NASA POWER parameter dictionary before use, because names are the one thing the agent must not guess. If the diffuse or direct component is not available hourly, estimate it from the global value with the Erbs correlation in pvlib and say so in the interface. Request pattern for the hourly point service:

```
https://power.larc.nasa.gov/api/temporal/hourly/point?parameters=T2M,RH2M,WS10M,PS,ALLSKY_SFC_SW_DWN,ALLSKY_SFC_SW_DIFF,ALLSKY_SFC_SW_DNI,ALLSKY_SFC_LW_DWN&community=RE&longitude=77.5771&latitude=34.1526&start=20240101&end=20241231&format=CSV&time-standard=UTC
```

- Set the time standard explicitly. The service offers UTC and local solar time; ask for UTC, compute solar position from it, and convert to Indian Standard Time (UTC+5:30) for display only.
- NASA marks missing values with a fill number (-999). The loader must flag them, never treat them as data, and say how many hours were affected.
- If the service limits the date range or number of parameters per request, split the request. Do not cut parameters silently.

- Default site is Leh: 34.1526 N, 77.5771 E, about 3,500 m. Use a complete recent calendar year for the bundled file.
- User CSV format: an ISO 8601 timestamp with timezone, then temperature in °C and global irradiance in W/m², with the other columns optional. The loader names the exact column and row when something is wrong.
- If the agent's sandbox cannot reach NASA, do not invent a Leh file. Build the loader and tests against a tiny fixture, develop the interface against a clearly labelled synthetic series, and ask the team to run scripts/fetch_weather.py. The interface must print the weather provenance on screen at all times.

4.2 Materials and glazing

The ranges below are starting points for the agent to check against real sources, not citations. The library file (data/materials.csv) has a source column for every row, filled in from the actual book, standard or datasheet, and a flag saying whether the value is typical or measured. Use the ASHRAE Handbook of Fundamentals (thermal properties of building materials), CIBSE Guide A, Indian references such as IS 3792 and the ECBC, and manufacturer datasheets. Any row without a source is not allowed in the file.

Material	λ (W/mK)	ρ (kg/m ³)	c (J/kgK)	Note
Dense concrete	1.4 to 1.75	2200 to 2400	840 to 1000	heavy, stores heat well
Fired clay brick	0.6 to 0.9	1700 to 1900	about 800	
Stone (granite, sandstone)	1.7 to 3.5	2300 to 2700	790 to 900	range is wide, pick the actual stone
Rammed earth	0.6 to 1.2	1700 to 2000	900 to 1000	local material
Adobe	0.5 to 0.7	1400 to 1800	about 900	local material
EPS insulation	0.032 to 0.040	15 to 30	about 1400	
XPS insulation	0.029 to 0.036	30 to 40	about 1450	
Mineral wool	0.035 to 0.045	30 to 100	840 to 1030	
PUR / PIR foam	0.022 to 0.028	30 to 45	about 1400	
Softwood timber	0.12 to 0.14	450 to 550	1600 to 2300	
Water (in containers)	0.6	1000	4180	sensible storage
Paraffin PCM	about 0.2	770 to 880	about 2000	latent heat about 180 to 220 kJ/kg, melting point chosen near the comfort band

Glazing (U in W/m²K, SHGC dimensionless), starting points to verify: single clear glass U about 5.7 and SHGC about 0.85; double clear U 2.7 to 3.0 and SHGC 0.70 to 0.76; double low-e with gas fill U 1.4 to 1.8 and SHGC 0.40 to 0.60. The night cover is described by its thermal resistance (default 0.8 m²K/W) and the hours it is closed.

5. Outputs, metrics and the optimiser

5.1 Metrics

Metric	Definition
Indoor temperature	Hourly indoor air temperature, free-floating run, for the chosen period.
Minimum and maximum	Lowest and highest indoor air temperature in the period.
Comfort hours	Hours with indoor air between the comfort low and high. Defaults 15 and 28 °C. The problem statement gives no number, so these are placeholders for the team to agree and state openly. Do not cite a standard for them.
Hours and degree-hours below the low limit	Count of hours below, and the sum of (low limit – temperature) over those hours, in K·h.
Overheating hours	Hours above the comfort high limit. Glazed shelters in strong sun can overheat, and ignoring it would favour oversized windows.
Daily swing	Maximum minus minimum indoor temperature for each day, and the mean.

Metric	Definition
Heating energy	kWh from the ideal heater run over the period, and per m ² of floor. The headline energy figure, labelled model estimate.
Solar energy	Incident on each facade and the roof (kWh/m ²), transmitted through glazing (kWh), absorbed by opaque surfaces (kWh).
Heat flow	Signed heat flow through each element (kWh, loss negative) and the instantaneous temperature difference between inside and outside.

5.2 Optimiser

The optimiser is a parameter sweep with a two stage search, not a black box. The variables below are the starting set. The agent checks that each range is physically sensible before using it.

Variable	Options
Wall build-up	Five or six stacks built from the library: stone, dense concrete, brick, rammed earth, timber frame, each with and without insulation
Insulation thickness (outside)	0, 50, 100, 150 mm
Orientation of the main facade	0 to 315° in 45° steps (180° is south)
Glazing area on the main facade	0, 5, 10, 15, 20, 30% of that facade
Glazing type	single clear, double clear, double low-e
Night cover	none, or R = 0.8 m ² K/W closed from late afternoon to mid morning
Thermal mass	none, 100 mm stone floor, water containers, 20 mm PCM layer on the inside face
Infiltration	0.2, 0.5, 1.0 air changes per hour
Plan aspect ratio at fixed floor area	1 : 1, 1.5 : 1, 2 : 1
Roof	flat, mono-pitch 15°

- **Stage 1, screening.** Run every candidate on two ten day windows of the chosen weather file: the coldest ten days by mean temperature, and the sunniest ten days of the warm season (to catch overheating). Include a three day warm-up.
- **Stage 2, confirmation.** Run the best twenty and the baseline for the full year.
- **Baseline.** Always include a plain reference design (for example 150 mm concrete block, single glazing on the main facade, no insulation) so the table shows each design relative to a named baseline. Show the difference in model heating energy. Never present it as a saving.
- **Score.** Heating energy in kWh, plus a user adjustable weight times overheating degree-hours. Defaults chosen so heating energy dominates, with the choice written down in the iteration log. Also show the trade-off as a scatter plot of heating energy against overheating degree-hours.
- **Reproducible.** Fixed random seeds, same results on every run. The sweep is a plain grid or a seeded Latin hypercube, whichever the agent shows to be faster for equal coverage.
- **Speed target.** One thousand designs on the ten day windows in under 60 seconds on a mid-range four core laptop. Profile first. Vectorise before reaching for compiled code; numba is allowed only if the Windows install is verified.

6. Verification and validation

A model that nobody has checked is a guess with equations. These tests are what lets the team say the solver is correct as software, even though it has not yet been compared with a measured shelter. Tests must be real checks against an independent answer (hand calculation, analytic solution, conservation law), not assertions that restate the code.

Test	What it checks	Pass condition
T1	Steady state U value: constant inside and outside temperature, no sun, fixed film coefficients	Heat flux equals $U \times \Delta T$ within 0.5%
T2	Energy balance on every run, including the heater	Boundary flows minus change in stored energy is within 0.5% of total flow
T3	Time step and grid convergence	Halving the step and the node spacing changes indoor temperature by under 0.2 K at every hour
T4	Solar closure and sense	Annual global irradiance in the solver equals the weather file within 1%. In the northern winter a south vertical surface receives more than a north one
T5	Analytic slab: one homogeneous layer, step change in outside temperature	Matches the series solution within 1%
T6	Lumped limit: thin, conductive slab	Matches the exponential decay with time constant RC
T7	Direction checks	More insulation never raises heating energy. Larger main facade glazing raises daytime solar gain. Turning the glazed facade away from the sun lowers winter gain
T8	Reproducibility and ranking	Same inputs give identical output. The best design beats the baseline on the stated score
T9	Portability	A weather file from another place runs with no code change
T10	Spin-up independence	Starting three days earlier changes results after the warm-up by under 0.05 K
T11 (stretch)	Cross-check of one reference case with EnergyPlus or with the ANSYS wall panel (section 7)	Differences reported and explained, not hidden

Field validation, later. When the team has a test box with sensors, scripts/validate.py reads the measured indoor and outdoor logs and prints RMSE, MAE, CV(RMSE) and NMBE with plots. ASHRAE Guideline 14 gives calibration limits for hourly data of NMBE within $\pm 10\%$ and CV(RMSE) within 30%. Quote those as pass criteria only after real measurements exist. Until then the honest statement is that the solver is verified against analytic and conservation checks and not yet validated against a shelter.

7. ANSYS bridge

The agent cannot run ANSYS, so it prepares everything up to the run and everything after it, and keeps the middle step simple enough for a student to do by hand.

Level A (required): wall panel check. scripts/export_ansys.py takes a design and a date window and writes three files. First, boundary_conditions.csv with time, outside air temperature, sol-air temperature, film coefficients and the indoor air temperature that the Python solver computed. Second, materials.csv with the layer properties actually used. Third, wall_panel.mac, an APDL macro that builds the exact layer stack as a thermal solid, applies the time-varying convection from the CSV, and writes the inner surface temperature and heat flux to a results file. Choose the element type from the ANSYS help for the installed version, because legacy element names change between releases; comment the macro line by line and say in its header that it has not been run by the agent. scripts/compare_ansys.py reads the ANSYS results and the Python wall node results and prints the maximum and RMS difference with a plot.

Level B (stretch): whole shelter. Build the shelter box in ANSYS Workbench transient thermal, with the indoor air as a lumped node, driven by the same exported tables, and compare indoor temperature over the coldest three days. This is the model the problem statement names. Do it only if Level A is done and the team has the licence time.

Needs the team. Run the macro or the Workbench model, export the results as CSV, run `compare_ansys.py`, and put the real difference on a slide. If the college licence is not available, the student edition has a node limit, so keep the mesh coarse. A small difference is a good result, a large one is a finding to explain, and both are better than saying the two were never connected.

8. AI slop: what it is, why it happens, how this project avoids it

How this section was produced. I searched for how designers and tool builders describe and detect AI-generated interfaces, and read several independent catalogues. Four of them are open-source design-skill repositories or documentation sites that list the patterns they check for (impeccable.style, the Hallmark pattern list, UI Craft, the ux-skill wiki), and two are blog posts that explain the cause (925 Studios and one on the purple gradient default). They agree closely with each other. None is peer reviewed research, so treat this as consolidated practitioner knowledge. The last row of the table, on code and documents, is my own addition and is not from those catalogues.

8.1 What it is and why every model makes it

AI slop in interface design is the generic look you get when a model fills in design decisions that nobody made. The shared explanation in the sources is that a language model returns the most common choice in its training data, and for web interfaces the most common choice since about 2019 is the Tailwind-era default: the Inter typeface, indigo and purple accents, rounded cards with soft shadows, a centred hero over three feature cards. One catalogue puts the test simply: if you cannot tell which product an interface belongs to, it is slop.

Two further points matter for how we instruct the agents. First, banning one default just moves the model to the next most common one. People report that after telling a model to avoid Inter it lands on Space Grotesk, and the catalogues now list Geist, Plus Jakarta Sans and Fraunces as overused too. So the rules below say what to do instead, with specific choices, not only what to avoid. Second, a model tends to remove features when asked to make a layout responsive, which is one of the signs of a lazy build, so the phone layout must keep every function.

The things you named (glowing neon, small text sitting above a bigger heading) are both on the list. The first is the dark background with a coloured glow around boxes. The second is the tiny uppercase, widely spaced label placed above a large headline, called the eyebrow, often with a status dot or a pill chip.

8.2 The tells, and what we do instead

Area	The tell	What we do instead
Typeface	Inter, Roboto, Arial, Open Sans, system-ui, and the newer defaults Geist, Plus Jakarta Sans, Space Grotesk, Fraunces. One family for everything.	IBM Plex Sans for interface text. IBM Plex Mono for numbers, units and axis labels. Source Serif 4 for the tool title and drawing titles only. Font files bundled with the app. The agent must not swap in anything from the banned list.
Colour	Purple to blue gradients, violet accents, cyan on dark, many accent colours, gradient text.	Warm off-white page, near-black ink, one warm accent for sun and heat, one cold accent for outside air, greys for the rest. No gradients anywhere. Palette in 8.4.
Surfaces and effects	Glass panels with blur, glowing borders, coloured shadows on dark backgrounds, a hairline border plus a wide soft shadow, a thick coloured stripe on one side of a card, radial blobs and stripes as decoration.	Flat panels. One pixel rules. No shadows, no blur, corner radius 0 to 3 px. Light theme only, so there is nothing to glow against.
Layout	Centred hero with a big button, three equal feature cards, same-size cards repeated, a small icon tile above each heading, a big number over a small label with supporting stats, every section the same height.	No landing page. The first screen is the working tool with the default Lehigh case already run. Layout follows the work: inputs on the left, results in the centre. Panels differ in size because their content does.
Type details	Tiny uppercase tracked label above a large heading, pill chips, a standard scale of 12/14/16/24/36/48, weight 600 on everything, lines as wide as the container, tight line height.	No eyebrows, no chips, sentence case labels. Scale in 8.4 with a clear ratio. Line height 1.4 to 1.5 for text, 60 to 75 characters per line. Numbers in tabular mono and right aligned.
Icons and imagery	Emoji as icons, several icon sets mixed, icons in coloured circles, stock illustration, fake browser or phone frames, 3D blobs.	No emoji, no illustration, no frames. At most a few plain inline icons from one set. Graphics are charts and drawings generated from the user's actual inputs.

Area	The tell	What we do instead
Motion	Drifting blobs, scale or rotate on hover, fade-in on scroll, bouncy easing.	No decorative motion. Hover changes colour only. Plots redraw without animation.
Copy	Em dashes throughout, words like seamless, powerful, leverage, unlock, empower, supercharge, intelligent, cutting-edge. Short aphorisms in the pattern of 'not just X, but Y'. Round invented numbers (10x faster, 99% accurate). Placeholder names.	Plain statements in the voice of a careful engineering student. Every number comes from the model or a cited source. No em dashes in any interface text, README or comment. Labels are nouns with units, such as 'Wall thickness (mm)'.
States and responsiveness	Only the happy path shown. No empty, loading or error states. Mobile layout that removes features.	Every panel has a real empty state saying what to do next, a loading state saying what is being computed, and errors that name the field and the value. On a phone the same functions work in one column.
Data	Fake dashboards with random numbers, decorative sparklines, stock chart shapes.	Charts come only from real runs. Synthetic data, when used for development, is announced in a visible banner on every screen.
Code and documents (my addition)	README with emoji headings, badge walls and feature lists. Comments that restate the code, a docstring on every trivial function, needless classes and layers, broad try/except that hides errors, TODO placeholders, tests that cannot fail.	A README with four short parts. Comments only for why. Plain functions. Errors raised with a specific message. Tests that compare with an independent answer.

8.3 Three tests the agent must run on its own screenshots

- **Swap test.** Imagine the same screenshot with a different project name on it. If nothing in the picture stops it belonging to some other product, it is slop. The VAJRA screen must visibly be about Leh weather, a shelter drawn to scale and heat flow in watts.
- **Five specifics.** List five things in the screenshot that only make sense for a shelter thermal tool. Examples: sunrise and sunset shading on the time axis, a comfort band, wall layers drawn to scale from the chosen materials, heat flow split by element, a weather source line with coordinates. Fewer than five means the screen is generic.
- **Squint test and delete test.** Blur the screenshot: the biggest shapes should be data, not decoration, and nothing should glow. Then remove any element in your head. If the screen is as useful without it, delete it.

8.4 Design specification for VAJRA

Palette. Every text colour must reach WCAG AA contrast (4.5 : 1) on its background. No other colours are allowed in the CSS or the charts.

Token	Hex	Use
paper	#F5F2EB	page background
panel	#FBFAF6	input column and table background
ink	#1E2125	text, indoor temperature line, drawing lines
muted	#5E6368	secondary text, units, axis labels
rule	#CFC8BA	one pixel dividers, table lines, grid
band	#E4DFD2	comfort band and night hours shading
sun	#B5521B	solar gain and heat input
cold	#2E5E7E	outside air temperature and heat loss
error	#A12B2B	error text only

Type. IBM Plex Sans 400 for text and 600 for panel headings only. IBM Plex Mono for every number, unit and axis label, with tabular figures. Source Serif 4 for the tool title and the drawing title block. Base size 15 px. Scale with a ratio of 1.25: 12 (axis and units), 15 (text), 18.75 (section headings), 23.4 (tool title). Nothing larger, because there is no hero section and no display text.

Layout. A fixed 300 px input column on the left. The centre holds the main plot at the top, then the shelter section drawing and the heat flow chart side by side, then the comparison table across the full width. At 390 px wide everything

stacks in the order inputs, plot, drawing, heat flow, table, with no feature removed. Table rows 32 px, numeric columns right aligned with the unit in the header, sortable by any column. Focus rings 2 px ink, all fields labelled, keyboard operable.

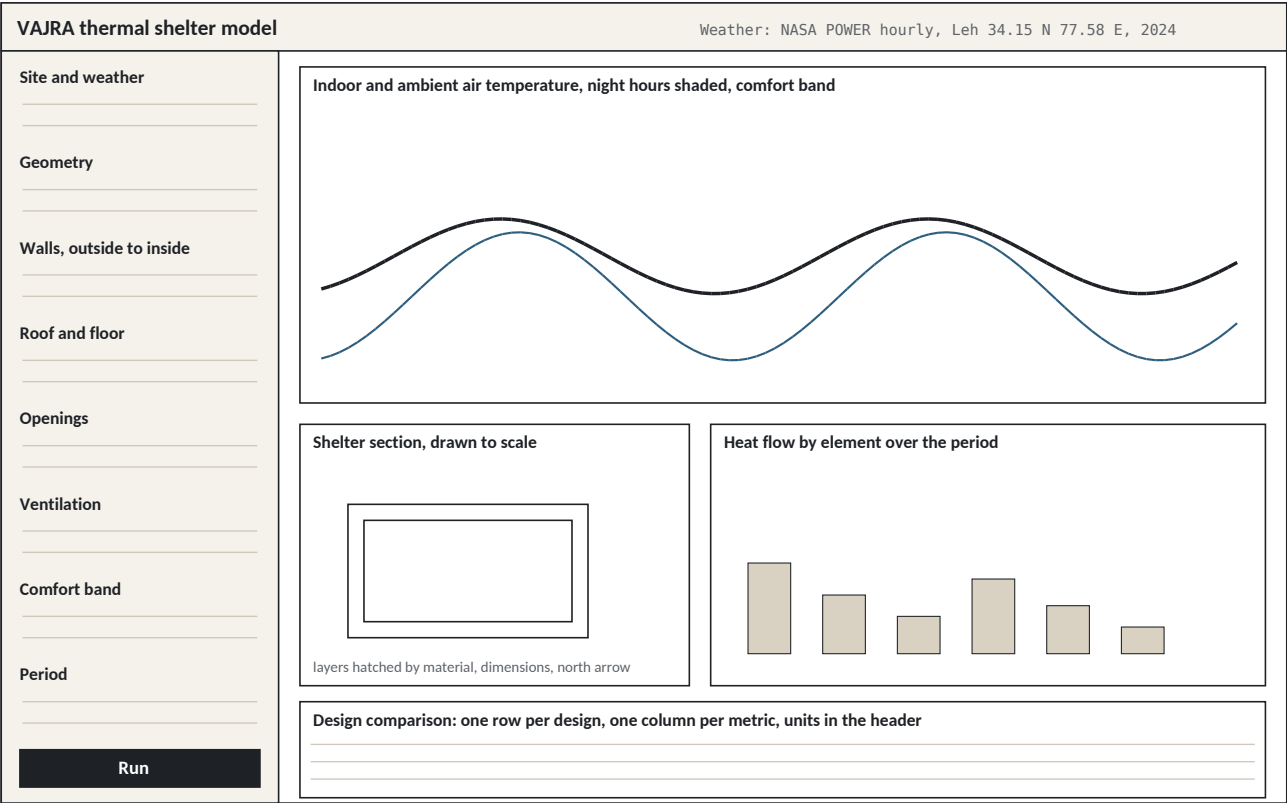


Figure 2. Layout wireframe at 1440 by 900. The curves and bars here are placeholders. The real interface plots only results from the solver.

Charts. Plotly.js from a local file, restyled by hand: modebar reduced to the image download, grid lines in rule colour at 1 px, text in Plex. Indoor temperature is a 2 px ink line, outside air a 1.5 px cold line, solar irradiance a thin sun-coloured area. The comfort band and the night hours are shaded in the band colour, with night derived from the actual sunrise and sunset for the date and place. Lines are labelled directly at their ends instead of with a legend box. No markers, no gradient fills, no chart titles that repeat the panel heading. The minimum temperature point gets a plain text annotation with its value and time.

Shelter section drawing (the signature element). Generated as SVG from the actual design, to scale, in the manner of an architectural section. Each layer is hatched by material using the usual drawing conventions: diagonal lines for masonry, dots with small triangles for concrete, zigzag for insulation, lines along the grain for timber, short ticks for soil, wavy horizontal lines for water, a light crosshatch for PCM. Line weights: outline 1.2 px, layer boundaries 0.7 px, hatching 0.4 px. Dimension lines with ticks give thickness in mm and height in m. Leader lines carry the material names in Plex Mono 11 px. A north arrow, a scale bar, and the solar noon sun angle for the selected day sit beside the section. A small title block at the bottom right gives the design name, scale, date and weather source, as on an engineering drawing. This is the one place the interface gets a distinctive look, and it is useful because it shows what the user entered.

Copy. Say what the thing does and nothing more. Bad: 'Supercharge your shelter design with intelligent insights.' Good: 'Run the design on the weather file below.' Bad: 'Something went wrong.' Good: 'Thickness of layer 2 must be between 5 and 500 mm (got 0).' The empty state of the results panel reads 'No run yet. Choose a design and press Run.' The default view loads the bundled Leh case already run, so nobody sees an empty page first.

Design sheet. A4 export of the chosen design on a white background, using the same type, with the section drawing, the inputs, the metrics table, the weather source and a line stating that values are model estimates not yet compared with a measured shelter.

8.5 Automated scan (scripts/slop_scan.py)

The build fails if the scan finds any of the following in the files under web/ or in rendered output. The scan catches the lazy tells. It does not replace the visual review in 8.3, because a scan cannot judge taste.

- Any CSS gradient function, any *backdrop-filter* or *filter: blur*.
- Any *box-shadow* at all, or any shadow with a colour or a blur above 4 px.
- *text-transform: uppercase* anywhere, or *letter-spacing* above 0.02 em.
- *border-radius* above 4 px.
- A first font family outside IBM Plex Sans, IBM Plex Mono, Source Serif 4, or any font or script loaded from an http or https address.
- Any colour outside the palette tokens, and any colour whose hue falls between 250° and 320° (the purple range).
- *@keyframes*, or any transition longer than 150 ms.
- The em dash character or a double hyphen used as a dash in templates, README, comments or interface strings.
- Emoji code points in interface strings.
- Any of these words in interface text: seamless, leverage, unlock, empower, supercharge, cutting-edge, revolutionary, intelligent, powerful, robust, next-generation, world-class, effortless.

9. Repository layout, data contract, milestones

9.1 Layout

```
vajra/  
  README.md  pyproject.toml  run.py  run.bat  
  data/weather/      leh_<year>_hourly.csv (fetched by the team), fixture_*.csv (tests only)  
  data/materials.csv  one row per material, with a source column  
  vajra/             weather.py solar.py materials.py geometry.py solver.py  
                     metrics.py optimise.py report.py ansys_export.py api.py  
  web/               index.html app.css app.js drawing.js fonts/ vendor/plotly.min.js  
  scripts/           fetch_weather.py slop_scan.py screenshots.py validate.py compare_ansys.py  
  tests/             test_t01 ... test_t11  
  docs/              contract.md iteration-log.md
```

Python 3.11 or newer. Allowed dependencies: numpy, scipy, pandas, pvlib, fastapi, uvicorn, pytest, and playwright for screenshots (development only). The interface is plain HTML, CSS and JavaScript with no build step and no node. The deck says Streamlit and Plotly for the dashboard. A FastAPI back end with a plain page gives much better control over the look, which is the point of section 8, and Plotly stays in the chart layer, so the deck remains accurate if the team says 'web dashboard'. If the team prefers Streamlit anyway, every rule in section 8 still applies and the agent must override Streamlit's default theme and layout to meet it.

9.2 Data contract

The agent writes this into docs/contract.md at the start, so that the solver, the interface and the exporters can be built in parallel. The example is illustrative. Comments are not part of the real JSON, and the zeros in the Result are placeholders that show the shape only.

```

Design
{ "name": "baseline_block",
  "site": {"lat": 34.1526, "lon": 77.5771, "elevation_m": 3500},
  "geometry": {"length_m": 4.0, "width_m": 3.0, "wall_height_m": 2.4,
    "roof": {"type": "mono_pitch", "pitch_deg": 15, "high_side": "N"},
    "azimuth_deg": 180}, // direction the glazed facade faces
  "constructions": { // layers listed outside to inside
    "wall": [{"material": "stone_masonry", "mm": 300}],
    "roof": [{"material": "mineral_wool", "mm": 100}, {"material": "timber", "mm": 25}],
    "floor": [{"material": "concrete_dense", "mm": 100}],
    "ground": {"soil_depth_m": 2.0} },
  "openings": [{"facade": "main", "width_m": 1.8, "height_m": 1.2, "glazing": "double_clear",
    "night_cover": {"r_m2k_per_w": 0.8, "from_hour": 17, "to_hour": 9}}],
  "air": {"infiltration_ach": 0.5, "vent_ach": 0.0, "vent_hours": [11, 15]},
  "internal_gains_w": 0,
  "solar_split": {"floor_and_mass": 0.7},
  "comfort": {"low_c": 15, "high_c": 28}, "setpoint_c": 15 }

Weather frame (one row per hour, UTC index)
t_air_c, rh_pct, wind10_ms, pressure_kpa, ghi_wm2, dhi_wm2, dni_wm2, lw_down_wm2
plus meta: source, lat, lon, elevation_m, synthetic (true or false), missing_hours

Result (shape only)
{ "meta": {"weather_source": "...", "synthetic": false, "period": ["...", "..."], "dt_minutes": 10},
  "series": {"time": [], "t_air_out_c": [], "t_air_in_c": [], "ghi_wm2": [], "solar_in_w": [],
    "q_w": {"wall_main": [], "wall_back": [], "wall_left": [], "wall_right": [],
      "roof": [], "floor": [], "glazing": [], "ventilation": []},
    "heater_w": []},
  "summary": {"t_min_c": 0, "t_max_c": 0, "comfort_hours": 0, "hours_below_low": 0,
    "degree_hours_below_kh": 0, "overheat_hours": 0, "heating_kwh": 0,
    "solar_incident_kwh_m2": {}, "solar_transmitted_kwh": 0,
    "solar_absorbed_opaque_kwh": 0, "flow_totals_kwh": {}} }

```

9.3 Milestones and gates

Build in this order. Do not start a milestone until the previous gate passes, except that the interface (M4) may start against a mock of the API once M0 is done. If several agents work in parallel, the lead owns docs/contract.md and the merges. Suggested tracks: A solver and tests (M0, M1, M3), B data and materials (M2), C interface and drawing (M4), D exporters and docs (M5, M6).

	Deliverable	Gate
M0	Walking skeleton: loader on a fixture, single wall and one air node, one API endpoint, a plain page that plots indoor and ambient temperature. python run.py starts it.	End to end run works. T1 passes on the skeleton.
M1	Full physics from section 3: layers, PCM, solar on every surface, windows, ventilation, ground, ideal heater run.	T1 to T6 and T10 pass.
M2	Weather and materials: fetch script, loader with clear errors, materials.csv with a source on every row, custom materials.	Every material row has a source. Bad CSV gives a precise message. T9 passes.
M3	Metrics, optimiser, compare endpoint, baseline row.	T7 and T8 pass. Speed target met or the shortfall reported.
M4	Interface built to section 8: inputs, plot, section drawing, heat flow chart, comparison table, design sheet, empty, loading and error states.	slop_scan passes. Screenshots at 1440 and 390 reviewed with the three tests. All functions work at 390.
M5	ANSYS export Level A and compare script.	Files generate, unit tests on their text pass, hand-off note written for the team.
M6	README, three minute demo script with the exact click path, a page listing what the tool does not do, fresh clone test.	A clean virtual environment on Windows paths installs and runs offline. Final report delivered.

10. Iteration protocol and stop rules

The aim is a good result quickly, not endless polishing. The loop below is what the agent repeats, and the stop rules say when it must hand over.

- **Smallest slice first.** Pick the smallest piece of work that moves a gate.
- **Measure, do not guess.** When a choice is open, run the alternatives and keep the better one with the numbers logged. Experiments worth running: time step 60, 30 and 10 minutes; node spacing; isotropic against Hay-Davies sky model; apparent capacity against enthalpy for the PCM; layout and type size alternatives judged on screenshots.
- **Look at the screen.** For every interface change, render screenshots at 1440 by 900 and 390 by 844, open the images, run the scan, and apply the three tests in 8.3.
- **Log it.** One short entry in docs/iteration-log.md: what changed, why, the evidence.
- **Severity.** High: a wrong number, a crash, or a tell from 8.2. Medium: a confusing label or a layout break at a tested width. Low: taste.
- **Stop when** every gate has passed and two review passes in a row find nothing high or medium. After six polish cycles on the interface, stop regardless and list what remains.
- **Blocked.** After two failed attempts, write down the blocker and move to the next gate.
- **Scope.** No features outside this brief without asking.

What only the team can do

- Run scripts/fetch_weather.py on a machine with internet and put the file in data/weather.
- Run the exported case in ANSYS and send back the results file.
- Agree the comfort band and say it out loud in the presentation.
- Supply datasheets for any locally available material to add to the library.
- Later: build a small test box with sensors and log it, so validate.py has real data.

11. Master prompt

Paste this into the lead agent together with this PDF. The same text is in VAJRA_master_prompt.txt, which is the better file to copy from.

You are the lead engineer for VAJRA, a hackathon prototype. You plan, build, test and polish it end to end. You may split the work between sub-agents; you own integration and the final result.

Before writing any code, read VAJRA_build_brief.pdf completely. It is the specification. If this prompt and the brief disagree, this prompt wins.

WHAT YOU ARE BUILDING

A working tool for Smart India Hackathon problem statement PS26051, "Software based model development for design of area specific shelter for thermal comfort maintenance". The user gives hourly weather for a place (default: Leh, Ladakh) and describes a shelter: size, shape, orientation, wall, roof and floor layers, thermal mass or phase change material, windows and ventilation. The tool predicts hourly indoor air temperature, the solar energy gained, and the heat flow through each part of the shelter. It compares many designs on the same weather and ranks them by how little heating energy they need to stay comfortable. It exports the best design as input for an ANSYS transient thermal check. The core is a physics solver in Python. Do not add machine learning.

RULES THAT DO NOT BEND

1. Real data only. Weather comes from NASA POWER (hourly) or a CSV the user supplies. Material properties come from published sources, with the source written in the data file next to each value. If your sandbox cannot reach NASA, build and test the loader, develop against a synthetic test series that is labelled "synthetic test data" in the interface whenever it is on screen, and tell me to run scripts/fetch_weather.py. Never show synthetic output as a result for Leh.
2. No accuracy, savings, payback or cost claims anywhere (interface, README, comments, design sheet). The model has not been compared with a measured shelter. Report what it computes and label it "model estimate".
3. It must not look AI-made. Section 8 of the brief is a hard specification: palette, type, layout, drawing conventions, copy rules and the automated scan. A build that fails scripts/slop_scan.py is not finished. Do not substitute fonts, colours or layouts from your own defaults. If you notice yourself reaching for a gradient, a glow, a card grid, an icon above a heading, a small uppercase label above a large one, or an em dash, stop and use the brief instead.
4. It runs offline on a Windows laptop with Python 3.11 or newer, started by one command (python run.py). No CDN links at runtime. Fonts and the Plotly.js file are bundled in the repository. Use pathlib and avoid shell-specific scripts, apart from an optional run.bat.
5. Be exact about what is not done. Anything unfinished or untested goes in the final report, not under the carpet. Do not write tests that pass by construction.
6. Stay inside the brief. Do not add features, pages or dependencies it does not call for without asking.

HOW TO WORK

Write docs/contract.md first, using the Design, Weather and Result data shapes from section 9 of the brief, so that parallel work can proceed against a fixed interface. Then build in this order

and do not skip ahead: M0 walking skeleton, M1 complete physics, M2 data and materials, M3 metrics and optimiser, M4 interface, M5 ANSYS export, M6 docs and hardening. Each milestone has a gate in the brief. Do not start the next milestone until the gate passes, except that interface work (M4) may start against a mock of the API once M0 is done. Get M0 working end to end first, even if it is ugly, because integration problems show up there.

THE LOOP (repeat for every slice of work)

1. Pick the smallest slice that moves a gate.
2. Implement it.
3. Run the tests. Fix failures before anything else.
4. If a design choice is open (time step, grid spacing, sky model, PCM method, layout, type size), run the alternatives, record the numbers or screenshots in docs/iteration-log.md, and keep the better one. Do not decide by taste when you can measure.
5. For anything visual: render screenshots with scripts/screenshots.py at 1440 by 900 and 390 by 844, open the images and look at them. Run scripts/slop_scan.py. Apply the swap test, the five specifics test and the squint test from section 8.3. Fix what fails.
6. Add a short factual entry to the iteration log: what changed, why, what evidence.
7. Commit with a plain one-line message.

STOP RULES

Stop polishing when all milestone gates pass and two consecutive review passes find nothing of high or medium severity (high: a wrong number, a crash, or a tell from the brief; medium: a confusing label or a layout break at a tested width). If that has not happened after six polish cycles on the interface, stop anyway, deliver, and list what remains. If you are blocked on a task after two attempts, write the blocker down and move to the next gate.

WHAT YOU MAY ASK THE TEAM FOR (and only these)

Running scripts/fetch_weather.py on a machine with internet. Running the exported case in ANSYS. Agreeing the comfort band (the default of 15 to 28 degrees C is a placeholder, not a standard). Datasheets for any locally available material the team wants in the library.

WRITING STYLE FOR EVERYTHING A HUMAN WILL READ

README, interface text, help text, comments, commit messages: plain sentences, the way a careful engineering student writes. No em dashes. No emoji. No marketing words (seamless, powerful, leverage, unlock, empower, supercharge, intelligent, robust, cutting-edge). Comments explain why, never what. The README has four parts: what it is, how to run it, how to test it, what it does not do yet.

FINAL REPORT (when you stop)

1. How to start it, in two lines.
2. Gate status table: each milestone, passed or not, with the test names.
3. Verification results: the actual numbers from tests T1 to T10.
4. What is untested or assumed (for example the 70/30 split of transmitted solar, ground properties, glazing values).
5. Remaining issues, ordered by severity.
6. What the team must do next.

Do not summarise with praise. State the facts.

12. Sources

On AI slop in interface design (read in October 2026):

- impeccable.style/slop and impeccable.style/anti-patterns: catalogue of 37 patterns marking an interface as AI-generated, with detection rules.
- mintlify.com/pbakaus/impeccable/concepts/anti-patterns: the same author's explanation of the anti-patterns and the AI slop test.
- 925studios.co/blog/ai-slop-design-tells: the typeface and gradient tells, and why models return the training median.
- github.com/Laith0003/ux-skill, wiki page 'How to detect AI slop in your design': eight categories of fingerprints, including type scale, content and empty states.
- instagit.com, Hallmark repository: named anti-pattern list, including eyebrow labels, invented metrics, icon tiles and mixed icon sets.
- typingmind.com/skills/educlopez-ui-craft and skills.cat (atris design skill): rules on sentence case, em dashes, single icon set, no glassmorphism.
- prg.sh, 'Why your AI keeps building the same purple gradient website': the training data explanation for the purple default.

On the weather data: NASA POWER hourly API documentation

(power.larc.nasa.gov/docs/services/api/temporal/hourly/) and parameter dictionary (power.larc.nasa.gov/parameters/).

Technical references to look up while building: ISO 6946 (surface resistances and U value calculation), ASHRAE Handbook of Fundamentals (material properties, solar and heat transfer), ASHRAE Guideline 14 (calibration criteria), pvlib documentation (solar position and irradiance transposition), ANSYS Mechanical APDL documentation for the installed version.